

Coupled Vehicle Dynamics, Powertrain, and Tyre Thermodynamics Formulation for Predictive Race Intelligence

Tactical overtaking decisions in modern hybrid motorsport cannot treat electrical energy deployment and tyre grip as decoupled operational budgets¹. Evaluating whether an attack window yields a net race advantage requires resolving an instantaneous multi-physics trade-off: tractive bursts delivered by high-power hybrid drives induce severe battery polarization, ohmic heating, and contact-patch shear that directly accelerate electrochemical ageing and thermal tyre degradation². A strategy tool that optimizes for immediate spatial delta without modeling these non-linear physical constraints risks precipitating premature battery thermal derating, exceeding regulatory energy recovery ceilings, or pushing tyre surface temperatures into severe friction decay².

The single-car physical architecture developed here establishes a mathematically rigorous foundation for predictive race engineering¹. The vehicle is formulated as a coupled dynamic system comprising a curvilinear Frenet-frame chassis model, a 2026 FIA-compliant hybrid energy recovery system with Manual Override Mode (MOM) logic, a 1-RC Thevenin equivalent circuit battery model paired with a two-node core-surface thermal network, and a dual-layer thermo-mechanical tyre friction and wear model¹. Operating deterministically at 100 Hz, this formulation captures the second-order coupling between electrical power draw, voltage sag, thermal dissipation, contact-patch combined slip, and cumulative lap-time pace degradation¹.

Curvilinear Kinematics and Longitudinal Road-Load Dynamics

Chassis motion is resolved along the longitudinal body axis and projected onto a curvilinear Frenet-Serret reference frame defined by the circuit centerline geometry¹. The vehicle mass translation obeys Newton's second law, balancing contact-patch propulsion and friction braking forces against aerodynamic drag, rolling resistance, and gravitational track gradient effects¹:

$$m_{\text{veh}} \dot{v}_x = F_{x,\text{total}} - F_{\text{drag}} - F_{\text{rr}} - m_{\text{veh}} g \sin \theta$$

Aerodynamic performance varies dynamically according to active aerodynamic configurations mandated by contemporary regulations, switching between high-downforce cornering trim (Mode Z) and low-drag straight-line trim (Mode X)⁶:

$$F_{\text{drag}} = \frac{1}{2} \rho_{\text{air}} C_d(\text{mode}) A_{\text{ref}} v_x^2$$

$$F_{\text{downforce}} = \frac{1}{2} \rho_{\text{air}} C_L(\text{mode}) A_{\text{ref}} v_x^2$$

The normal load distribution dictates instantaneous vertical tyre forces across axles, incorporating static gravity components, aerodynamic downforce, and dynamic longitudinal load transfer induced by acceleration and braking¹:

$$F_{z,\text{total}} = m_{\text{veh}} g \cos \theta + F_{\text{downforce}}$$

$$F_{z,\text{front}} = \frac{l_r}{L} m_{\text{veh}} g \cos \theta + \frac{l_r}{L} F_{\text{downforce}} - \frac{h_{\text{cg}}}{L} m_{\text{veh}} \dot{v}_x$$

$$F_{z,\text{rear}} = \frac{l_f}{L} m_{\text{veh}} g \cos \theta + \frac{l_f}{L} F_{\text{downforce}} + \frac{h_{\text{cg}}}{L} m_{\text{veh}} \dot{v}_x$$

Rolling resistance accounts for vertical tyre deflection and hysteretic rubber losses as a function of total vertical load:

$$F_{\text{rr}} = C_{\text{rr}} F_{z,\text{total}}$$

To track progress along complex track layouts, Cartesian global coordinates (x, y, ψ) are mapped into curvilinear Frenet coordinates $(s, n, \Delta\psi)$, where s denotes cumulative arc length along the track centerline, n represents orthogonal lateral track deviation, and $\Delta\psi = \psi - \psi_{\text{track}}(s)$ represents the vehicle heading error relative to the local track tangent¹:

$$\dot{s} = \frac{v_x \cos \Delta\psi - v_y \sin \Delta\psi}{1 - n\kappa(s)}$$

$$\dot{n} = v_x \sin \Delta\psi + v_y \cos \Delta\psi$$

$$\dot{\psi} = r$$

$$\dot{\Delta\psi} = r - \kappa(s)\dot{s}$$

Here, $\kappa(s)$ denotes the local horizontal track curvature¹. When evaluating interactions against a tracked competitor, relative progress gaps and closing rates are established directly through Frenet differentials²:

$$g_s = s_{\text{opp}} - s_{\text{ego}}$$

$$c_s = -\dot{g}_s = \dot{s}_{\text{ego}} - \dot{s}_{\text{opp}}$$

$$\Delta v = v_{x,\text{ego}} - v_{x,\text{opp}}$$

$$\Delta\psi_{\text{gap}} = \psi_{\text{ego}} - \psi_{\text{opp}}$$

Regulatory Powertrain Architecture and Kinetic Energy Recovery Constraints

The powertrain architecture models the 2026 FIA technical regulations, where the Motor Generator Unit - Kinetic (MGU-K) power output is elevated to 350 kW, working alongside a 400 kW turbocharged Internal Combustion Engine (ICE), while the thermodynamic MGU-H is eliminated⁵. Total propulsive force delivered to the drive wheels balances mechanical and electrical power contributions across the transmission¹:

$$F_{x,\text{prop}} = \frac{P_{\text{ice}}\eta_{\text{ice}} + P_{\text{mguk, mech}}\eta_{\text{gb}}}{v_x}$$

Powertrain Parameter	Baseline Standard Mode	Manual Override Mode (MOM)	Regulatory Reference and Governing Rule
MGU-K Peak Power	350 kW	350 kW	FIA Technical Regulations Article C5.2 ⁸
Full-Power Cutoff	290 km/h (80.56	337 km/h (93.61 m/s)	Speed-dependent

Velocity	m/s)		electrical derating curve ⁶
Power Output at 345 km/h	0 kW	~200 kW (~270 bhp)	Tactical overtaking power surplus margin ⁶
Zero-Power Cutoff Velocity	345 km/h (95.83 m/s)	355 km/h (98.61 m/s)	Complete electrical deployment cessation ⁶
Maximum Kinetic Recharge	8.5 MJ per lap	8.5 MJ per lap	FIA Technical Regulations Article C5.2.10 ⁵

Under standard operational deployment, electrical tractive power tapers at high speeds to prevent early depletion of the Energy Store and mitigate terminal straight-line divergence⁶. When tracking within one second of a competitor at defined detection checkpoints, the car unlocks Manual Override Mode (MOM), which sustains full 350 kW deployment across extended velocity windows⁶:

Standard Mode Deployment Profile:

- For vehicle speeds up to 290 km/h (80.56 m/s), maximum power is maintained at 350 kW⁶.
- Between 290 km/h and 339 km/h, power decreases linearly from 350 kW down to 105 kW⁶.
- Between 339 km/h and 345 km/h, power drops steeply from 105 kW to 0 kW⁶.
- Above 345 km/h, electrical deployment ceases entirely⁶.

Manual Override Mode Profile:

- For vehicle speeds up to 337 km/h (93.61 m/s), maximum power is sustained at 350 kW⁶.
- Between 337 km/h and 355 km/h, power tapers linearly from 350 kW down to 0 kW⁶.
- Above 355 km/h, electrical deployment ceases entirely⁶.

Braking demands initiate torque blending between front friction calipers, rear friction calipers, and rear-axle regenerative braking¹. The commanded mechanical regeneration power is constrained by motor capacity, inverter throughput, battery charge acceptance, and rear tyre contact-patch adhesion limits¹:

$$P_{\text{regen, mech}} = \min \left(|F_{x,\text{brake}}| \beta_{\text{bias}} v_x, P_{\text{mguk, cap}}, \frac{P_{\text{bat, charge, max}}}{\eta_{\text{inv}}}, \mu_{\text{rear}} F_{z,\text{rear}} v_x \right)$$

Delivered regenerative electrical power is integrated over each lap:

$$E_{\text{lap, regen}}(t) = \int_{t_{\text{lap start}}}^t P_{\text{regen, elec}}(\tau) d\tau \leq 8.5 \text{ MJ}$$

If cumulative lap recovery reaches the 8.5 MJ regulatory ceiling, regenerative braking is disabled for the remainder of the lap, routing all subsequent braking torque through mechanical friction discs⁵.

Electrochemical Equivalent Circuit and Two-Node Core-Surface Battery Dynamics

The traction battery pack is modeled using a Thevenin 1-RC equivalent circuit network coupled with a two-node lumped thermal system, capturing internal ohmic losses, charge transfer polarization overpotentials, and internal thermal gradients³.

The electrical circuit consists of a bulk open-circuit potential connected in series with an internal ohmic resistance and an RC parallel branch representing diffusion polarization³. Pack terminal voltage is given by³:

$$V_t = V_{oc}(SOC, T_c) - I_{\text{pack}} R_0(SOC, T_c, SOH_R) - V_1$$

$$\dot{V}_1 = -\frac{V_1}{R_1 C_1} + \frac{I_{\text{pack}}}{C_1}$$

Pack State of Charge ($SOC \in [0, 1]$) is tracked via Coulomb counting, where the nominal ampere-hour capacity Q_{nom} degrades over operational cycles through capacity health (SOH_C)¹:

$$SOC(t) = SOC(t_0) - \int_{t_0}^t \frac{\eta_c I_{\text{pack}}(\tau)}{Q_{\text{nom}} \cdot SOH_C} d\tau$$

Coulombic efficiency η_c is 1.0 during discharge ($I_{\text{pack}} > 0$) and 0.985 during charge ($I_{\text{pack}} < 0$).

Closed-Form Quadratic Current Resolution and Voltage Sag Clipping

When the MGU-K demands mechanical power at the rotor shaft, inverter losses establish the required DC electrical terminal power P_{elec} ¹:

$$P_{\text{elec}} = \begin{cases} \frac{P_{\text{mguk, mech}}}{\eta_{\text{inv}}} & \text{for } P_{\text{mguk, mech}} \geq 0 \quad (\text{discharge}) \\ P_{\text{mguk, mech}} \cdot \eta_{\text{inv}} & \text{for } P_{\text{mguk, mech}} < 0 \quad (\text{regeneration}) \end{cases}$$

Equating terminal power to the product of terminal voltage and pack current yields¹⁵:

$$P_{\text{elec}} = V_t I_{\text{pack}} = ((V_{oc} - V_1) - I_{\text{pack}} R_0) I_{\text{pack}}$$

Defining effective potential $V_{\text{eff}} = V_{oc} - V_1$ transforms the balance into a standard quadratic equation¹⁵:

$$R_0 I_{\text{pack}}^2 - V_{\text{eff}} I_{\text{pack}} + P_{\text{elec}} = 0$$

The discriminant $\Delta = V_{\text{eff}}^2 - 4R_0 P_{\text{elec}}$ determines physical solvability¹⁵. The Jacobi maximum power transfer theorem defines the theoretical discharge power ceiling:

$$P_{\text{max, discharge}} = \frac{V_{\text{eff}}^2}{4R_0}$$

If the requested electrical power exceeds $P_{\text{max, discharge}}$, the request is clipped, and the current settles at $I_{\text{pack}} = V_{\text{eff}}/(2R_0)$ ¹⁵. For feasible power requests ($\Delta \geq 0$), the physical root is extracted¹⁵:

$$I_{\text{pack}} = \frac{V_{\text{eff}} - \sqrt{V_{\text{eff}}^2 - 4R_0 P_{\text{elec}}}}{2R_0}$$

The delivered current is bounded by terminal voltage thresholds ($V_{\text{min}}, V_{\text{max}}$) to prevent lithium plating during heavy regeneration or severe undervoltage during discharge¹:

$$I_{\text{pack, max, discharge}} = \min \left(\frac{V_{\text{eff}} - V_{\text{min}}}{R_0}, I_{\text{discharge, limit}} \right)$$

$$I_{\text{pack, max, charge}} = \max \left(\frac{V_{\text{eff}} - V_{\text{max}}}{R_0}, -I_{\text{charge, limit}} \right)$$

Coupled Two-Node Core-Surface Thermal Dynamics

Under intense hybrid cycling, significant thermal gradients develop between the active internal

jelly roll (core) and the exterior cell housing (surface)³. The core temperature T_c and casing surface temperature T_s evolve through coupled thermal conduction and convection³:

$$C_c \frac{dT_c}{dt} = \dot{Q}_{\text{gen}} - \frac{T_c - T_s}{R_c}$$

$$C_s \frac{dT_s}{dt} = \frac{T_c - T_s}{R_c} - \frac{T_s - T_{\text{cool}}}{R_u}$$

where C_c and C_s represent core and casing heat capacities, R_c is conductive thermal resistance, R_u is convective resistance to the active liquid cooling jacket, and T_{cool} is coolant temperature³.

Total volumetric heat generation $\dot{Q}_{\text{gen}}$ is modeled via the extended Bernardi energy balance, accounting for ohmic Joule dissipation, polarization overpotential loss, and reversible entropic potential shifts¹⁶:

$$\dot{Q}_{\text{gen}} = I_{\text{pack}}^2 R_0 + \frac{V_1^2}{R_1} - I_{\text{pack}} T_c \frac{\partial V_{oc}}{\partial T}$$

To prevent thermal runaway, a progressive derating multiplier scales maximum deployment power as core temperature approaches safety limits²:

$$k_{\text{thermal, derate}} = \text{clip} \left(1.0 - \frac{T_c - T_{\text{warn}}}{T_{\text{crit}} - T_{\text{warn}}}, 0.0, 1.0 \right)$$

Electrochemical Ageing and State of Health Fade

Battery degradation accumulates through capacity loss (SOH_C) and internal impedance growth (SOH_R)². Ageing is driven by cumulative charge throughput, C-rate severity, and Arrhenius thermal acceleration²:

$$\frac{d(SOH_C)}{dt} = -\frac{|I_{\text{pack}}|}{3600 \cdot 2N_{\text{life}} Q_{\text{nom}}} \left(1 + \gamma_I \left(\frac{|I_{\text{pack}}|}{I_{1C}} \right)^{\beta_I} \right) \exp \left(\frac{E_a}{R} \left(\frac{1}{T_{\text{ref}}} - \frac{1}{T_c + 273.15} \right) \right)$$

$$\frac{d(SOH_R)}{dt} = \zeta_R \left| \frac{d(SOH_C)}{dt} \right|$$

$$R_0(t) = R_{0,\text{clean}} \cdot SOH_R(t)$$

Combined-Slip Tyre Mechanics, Flash Thermodynamics, and Pacejka Formulations

Tyre shear force generation dictates vehicle trajectory and progress, linking tractive torque to pavement reaction forces¹.

Kinematic longitudinal slip ratio κ_{slip} and lateral slip angle α_{slip} are derived from wheel rotational speed and chassis body velocities⁷:

$$\kappa_{\text{slip}} = \frac{\omega_w R_e - v_x}{v_x}, \quad \alpha_{\text{slip}} = -\arctan\left(\frac{v_y}{|v_x|}\right)$$

Normal load sensitivity degrades the effective peak friction coefficient non-linearly under heavy downforce and dynamic load transfer⁴:

$$\mu_{x,0}(F_z) = \mu_{x,\text{nom}} \left(1 - k_{\text{load},x} \frac{F_z - F_{z,0}}{F_{z,0}} \right)$$

$$\mu_{y,0}(F_z) = \mu_{y,\text{nom}} \left(1 - k_{\text{load},y} \frac{F_z - F_{z,0}}{F_{z,0}} \right)$$

Pure slip forces follow the Pacejka Magic Formula structure⁷:

$$F_{x0} = D_x \sin(C_x \arctan(B_x \kappa_{\text{slip}} - E_x(B_x \kappa_{\text{slip}} - \arctan(B_x \kappa_{\text{slip}}))))$$

$$F_{y0} = D_y \sin(C_y \arctan(B_y \alpha_{\text{slip}} - E_y(B_y \alpha_{\text{slip}} - \arctan(B_y \alpha_{\text{slip}}))))$$

where peak force capacities are $D_x = \mu_x(F_z, T_s^{\text{tyre}}, w)F_z$ and

$D_y = \mu_y(F_z, T_s^{\text{tyre}}, w)F_z$ ⁷. In combined acceleration, braking, and cornering, tractive and cornering forces interact within a modified friction ellipse⁷:

$$F_x = F_{x0} \sqrt{1 - \left(\frac{F_y}{D_y}\right)^2}$$

$$F_y = F_{y0} \sqrt{1 - \left(\frac{F_x}{D_x}\right)^2}$$

Dual-Layer Tyre Thermodynamics and Flash Heating

Tyre temperature dynamics are discretized into a thin external tread surface layer (T_s^{tyre}), which responds rapidly to frictional sliding dissipation, and a bulk structural carcass layer (T_c^{tyre}) representing the belt and sidewall¹:

$$C_{\text{tread}} \frac{dT_s^{\text{tyre}}}{dt} = \dot{Q}_{\text{fric}} - \frac{T_s^{\text{tyre}} - T_c^{\text{tyre}}}{R_{\text{cond, tyre}}} - h_{\text{conv}}(v_x) A_{\text{patch}} (T_s^{\text{tyre}} - T_{\text{amb}})$$

$$C_{\text{carcass}} \frac{dT_c^{\text{tyre}}}{dt} = \dot{Q}_{\text{def, core}} + \frac{T_s^{\text{tyre}} - T_c^{\text{tyre}}}{R_{\text{cond, tyre}}} - h_{\text{rim}} (T_c^{\text{tyre}} - T_{\text{rim}})$$

Frictional heat generation $\dot{Q}_{\text{fric}}$ represents the mechanical energy dissipated through contact-patch sliding velocity vectors⁴:

$$\dot{Q}_{\text{fric}} = |F_x(\omega_w R_e - v_x)| + |F_y v_x \tan \alpha_{\text{slip}}|$$

Convective cooling scales with forward speed: $h_{\text{conv}}(v_x) = h_0 + h_1 v_x^{0.8}$.

Reversible Thermal Grip Scaling and Mechanical Wear Accumulation

Tyre compound friction varies with tread surface temperature according to a Gaussian operating window⁴:

$$\mu_{\text{thermal}}(T_s^{\text{tyre}}) = \exp\left(-\frac{(T_s^{\text{tyre}} - T_{\text{opt}})^2}{2\sigma_{\text{tyre}}^2}\right)$$

Irreversible mechanical wear accumulates proportionally to frictional sliding power and thermal severity¹:

$$\frac{dw}{dt} = \kappa_{\text{wear}} \cdot \dot{Q}_{\text{fric}} \cdot \left(\frac{T_s^{\text{tyre}}}{T_{\text{ref, tyre}}} \right)^{\gamma_{\text{wear}}}$$

Remaining tyre life fraction $L_{\text{tyre}} \in [0, 1]$ and degraded force capacity $D_{x,y}$ incorporate accumulated wear state $w \in [0, 1]$ ¹:

$$L_{\text{tyre}}(t) = \max(0.0, 1.0 - w(t))$$

$$D_{x,y} = \mu_{x,y,0}(F_z) \cdot \mu_{\text{thermal}}(T_s^{\text{tyre}}) \cdot (1 - \lambda_{\text{wear}} w(t)) \cdot F_z$$

The lap-time degradation penalty Δt_{lap} observed across stints is modeled by mapping accumulated wear and fuel load burn-off¹:

$$\Delta t_{\text{lap}}(w) = a_1 w + a_2 w^2 + \beta_{\text{fuel}}(m_{\text{fuel}}(t) - m_{\text{fuel, ref}})$$

Discrete Multi-Physics State-Space Pipeline and Integration Architecture

The multi-physics model is compiled into an explicit 17-state vector driven by a 5-element control vector at 100 Hz¹:

$$\mathbf{x} = [v_x, s, n, \Delta\psi, SOC, V_1, T_c^{\text{bat}}, T_s^{\text{bat}}, SOH_C, SOH_R, T_s^{\text{tyre}}, T_c^{\text{tyre}}, w, E_{\text{lap, regen}}, m_{\text{fuel}}, x, y]^T$$

$$\mathbf{u} = [F_{\text{throttle}}, F_{\text{brake}}, \delta_{\text{steer}}, u_{\text{MOM}}, u_{\text{aero}}]^T$$

Execution Stage	Subsystem Dynamic Layer	Primary Physical Inputs	Governing Mathematical Formulation	State Updates Generated
1. Kinematics & Road Load	Aerodynamics & Geometry	vx, aero mode, curvature	Quadratic aero drag, downforce, axle load transfer	F_drag, F_downforce, F_z axles ¹

2. Powertrain Arbitration	2026 MGU-K & Combustion Engine	vx, throttle command, MOM flag	Speed-tapered MGU-K envelope, thermal derating	P_ice demand, P_mguk demand ⁶
3. Electrical Resolution	Battery Thevenin 1-RC Network	P_mguk request, SOC, V1	Quadratic power root, voltage sag boundaries	I_pack, V_t, V1 dot, SOC dot ³
4. Thermal & Ageing	Coupled Electro-Thermal Model	I_pack, V1, T_core battery	Extended Bernardi energy balance, Arrhenius law	T_core dot, T_surf dot, SOH_C dot, SOH_R dot ³
5. Contact Patch Shear	Pacejka & Dual-Layer Tyre Model	Drive force, brake force, steer angle	Magic Formula, friction ellipse, sliding power	F_x, F_y, T_tread dot, T_carcass dot, wear dot ⁴
6. Numerical Integration	100 Hz Deterministic Kernel	Full continuous state derivative	Fixed-step explicit 4th-order Runge-Kutta	Updated discrete state vector x[k+1] ¹

Deterministic Fixed-Step Python Simulation Engine

The following Python class implements the complete multi-physics single-car simulation model.

The codebase executes at 100 Hz ($dt = 0.01 \text{ s}$), resolving the battery electrical quadratic equation, two-node thermal ODEs, 2026 MGU-K deployment limits, and tyre thermo-mechanical wear dynamics¹.

Python

```
"""
```

Deterministic Multi-Physics Single-Car Powertrain and Dynamics Engine.
Complies with 2026 FIA MGU-K deployment limits, 1-RC battery electrical dynamics, two-node core-surface thermal balances, and Pacejka combined-slip tyre wear.

"""

```
from dataclasses import dataclass
import numpy as np
```

```
@dataclass(frozen=True)
class VehicleParameters:
    # Chassis Mass & Geometry
    mass_empty: float = 768.0    # Minimum vehicle mass without fuel (kg)
    mass_driver: float = 80.0    # Driver mass (kg)
    wheelbase: float = 3.600    # Wheelbase (m)
    weight_dist_front: float = 0.46 # Static front axle mass fraction
    h_cg: float = 0.30          # Center of gravity height (m)

    # Active Aerodynamics
    rho_air: float = 1.225      # Ambient air density (kg/m^3)
    frontal_area: float = 1.40  # Reference frontal area (m^2)
    cd_mode_z: float = 1.10     # Mode Z cornering drag coefficient
    cd_mode_x: float = 0.65     # Mode X straight-line drag coefficient
    cl_mode_z: float = 3.20     # Mode Z high downforce coefficient
    cl_mode_x: float = 1.40     # Mode X low drag lift coefficient
    crr: float = 0.012         # Rolling resistance coefficient

    # Powertrain & 2026 Regulations
    p_ice_max: float = 400000.0 # ICE peak power (400 kW)
    eta_drivetrain: float = 0.96 # Transmission mechanical efficiency
    eta_inverter: float = 0.95  # Bidirectional inverter efficiency
    p_mguk_max: float = 350000.0 # MGU-K peak power (350 kW)
    e_regen_lap_max: float = 8.5e6 # FIA kinetic recovery limit per lap (8.5 MJ)

    # 1-RC Battery Pack
    q_cell_nominal_ah: float = 5.0 # Individual cell capacity (Ah)
    cells_in_series: int = 180     # Series cell count (approx. 750V nominal)
    cells_parallel: int = 6        # Parallel branch count (30 Ah total pack)
    c1_polarization: float = 2500.0 # Polarization capacitance C1 (F)
    r1_polarization: float = 0.015 # Polarization resistance R1 (Ohms)
    v_term_min: float = 550.0     # Discharge cutoff voltage (V)
    v_term_max: float = 850.0     # Regeneration overvoltage cutoff (V)

    # Battery Two-Node Thermal Architecture
    c_core_bat: float = 14500.0   # Core heat capacity (J/K)
    c_surf_bat: float = 4500.0    # Casing surface heat capacity (J/K)
```

```

r_core_surf_bat: float = 0.22 # Core-to-surface conductive resistance (K/W)
r_surf_cool_bat: float = 0.12 # Casing-to-coolant convective resistance (K/W)
t_coolant_bat: float = 35.0 # Active coolant loop temperature (degC)
t_bat_warn: float = 55.0 # Thermal derating threshold (degC)
t_bat_crit: float = 65.0 # Critical thermal shutdown temperature (degC)
dvoc_dt: float = -0.00032 # Entropic coefficient dVoc/dT (V/K)

```

```

# Pacejka & Tyre Thermodynamics

```

```

r_effective: float = 0.33 # Loaded tyre rolling radius (m)
mu_x_nominal: float = 1.75 # Peak longitudinal friction coefficient
mu_y_nominal: float = 1.80 # Peak lateral friction coefficient
k_load_sensitivity: float = 0.12 # Friction load sensitivity decay factor
fz_ref: float = 4000.0 # Nominal reference tyre normal load (N)

```

```

c_tyre_tread: float = 3800.0 # Tread surface heat capacity (J/K)
c_tyre_carcass: float = 9500.0 # Bulk carcass heat capacity (J/K)
r_tyre_cond: float = 0.18 # Tread-to-carcass thermal conduction (K/W)
t_tyre_opt: float = 100.0 # Optimum thermal grip peak (degC)
tyre_thermal_sigma: float = 22.0 # Thermal operating window bell-curve spread
kappa_wear_rate: float = 4.2e-8 # Mechanical wear accumulation rate
gamma_wear_temp: float = 1.35 # Thermal wear acceleration exponent

```

```

class SingleCarEnergyModel:

```

```

    """

```

```

    Deterministic 100 Hz multi-physics simulation kernel modeling vehicle
    kinematics, 2026 MGU-K logic, battery electro-thermal dynamics, and tyre wear.

```

```

    """

```

```

    def __init__(self, params: VehicleParameters = VehicleParameters(), dt: float = 0.01):
        self.p = params
        self.dt = dt
        self.reset()

```

```

    def reset(self, soc_init: float = 0.85, fuel_init_kg: float = 70.0):

```

```

        # Kinematic and Spatial Coordinates

```

```

        self.vx: float = 15.0
        self.s: float = 0.0
        self.n: float = 0.0
        self.yaw: float = 0.0
        self.heading_error: float = 0.0
        self.x_map: float = 0.0
        self.y_map: float = 0.0
        self.lap_count: int = 1

```

```
self.fuel_kg: float = fuel_init_kg
```

```
# Battery States
```

```
self.soc: float = float(np.clip(soc_init, 0.0, 1.0))
```

```
self.v1: float = 0.0
```

```
self.t_core_bat: float = 40.0
```

```
self.t_surf_bat: float = 38.0
```

```
self.soh_capacity: float = 1.0
```

```
self.soh_resistance: float = 1.0
```

```
self.e_lap_regen: float = 0.0
```

```
# Tyre States
```

```
self.t_tyre_tread: float = 85.0
```

```
self.t_tyre_carcass: float = 80.0
```

```
self.tyre_wear: float = 0.0
```

```
# Telemetry Cache
```

```
self.p_deploy_delivered: float = 0.0
```

```
self.p_regen_delivered: float = 0.0
```

```
self.current_demand: float = 0.0
```

```
self.current_delivered: float = 0.0
```

```
self.voltage_terminal: float = 750.0
```

```
self.delta_energy_wh: float = 0.0
```

```
self.derate_factor: float = 1.0
```

```
def compute_open_circuit_voltage(self, soc: float, t_core: float) -> float:
```

```
    soc_c = float(np.clip(soc, 0.0, 1.0))
```

```
    v_cell_ref = (
```

```
        3.40
```

```
        + 0.70 * soc_c
```

```
        - 0.35 * (soc_c ** 2)
```

```
        + 0.85 * (soc_c ** 3)
```

```
        - 0.45 * (soc_c ** 4)
```

```
    )
```

```
    v_cell = v_cell_ref + self.p.dvoc_dt * (t_core - 25.0)
```

```
    return float(v_cell * self.p.cells_in_series)
```

```
def compute_internal_resistance(self, soc: float, t_core: float) -> float:
```

```
    t_kelvin = t_core + 273.15
```

```
    arrhenius = np.exp(1200.0 * (1.0 / t_kelvin - 1.0 / 298.15))
```

```
    soc_factor = 1.0 + 0.35 * ((1.0 - soc) ** 2)
```

```
    r_cell = 0.0018 * arrhenius * soc_factor
```

```
    r_pack = (r_cell * self.p.cells_in_series / self.p.cells_parallel) * self.soh_resistance
```

```
return float(r_pack)
```

```
def evaluate_2026_mguk_power_limit(self, speed_mps: float, manual_override: bool) -> float:
```

```
    speed_kmh = speed_mps * 3.6
```

```
    if manual_override:
```

```
        if speed_kmh <= 337.0:
```

```
            return self.p.p_mguk_max
```

```
        elif speed_kmh <= 355.0:
```

```
            taper = (355.0 - speed_kmh) / (355.0 - 337.0)
```

```
            return float(self.p.p_mguk_max * np.clip(taper, 0.0, 1.0))
```

```
        return 0.0
```

```
    else:
```

```
        if speed_kmh <= 290.0:
```

```
            return self.p.p_mguk_max
```

```
        elif speed_kmh <= 339.0:
```

```
            slope = (350000.0 - 105000.0) / (339.0 - 290.0)
```

```
            return float(350000.0 - slope * (speed_kmh - 290.0))
```

```
        elif speed_kmh <= 345.0:
```

```
            taper = (345.0 - speed_kmh) / (345.0 - 339.0)
```

```
            return float(105000.0 * np.clip(taper, 0.0, 1.0))
```

```
        return 0.0
```

```
def solve_battery_electrical(
```

```
    self, p_mech_request: float, v_oc: float, r0: float
```

```
) -> tuple[float, float, float]:
```

```
    v_eff = v_oc - self.v1
```

```
    if p_mech_request >= 0.0:
```

```
        p_elec_req = p_mech_request / self.p.eta_inverter
```

```
    else:
```

```
        p_elec_req = p_mech_request * self.p.eta_inverter
```

```
    # Power transfer feasibility limit
```

```
    p_transfer_limit = (v_eff ** 2) / (4.0 * r0)
```

```
    if p_elec_req > p_transfer_limit:
```

```
        p_elec_req = p_transfer_limit
```

```
        i_pack = v_eff / (2.0 * r0)
```

```
    else:
```

```
        discriminant = max(0.0, v_eff ** 2 - 4.0 * r0 * p_elec_req)
```

```
        i_pack = (v_eff - np.sqrt(discriminant)) / (2.0 * r0)
```

```
    # Voltage boundary clipping
```

```
    i_discharge_max = (v_eff - self.p.v_term_min) / r0
```

```

i_charge_max = (v_eff - self.p.v_term_max) / r0
i_delivered = float(np.clip(i_pack, i_charge_max, i_discharge_max))

v_terminal = float(v_eff - i_delivered * r0)
p_elec_delivered = v_terminal * i_delivered

if p_elec_delivered >= 0.0:
    p_mech_delivered = p_elec_delivered * self.p.eta_inverter
else:
    p_mech_delivered = p_elec_delivered / self.p.eta_inverter

return i_delivered, v_terminal, p_mech_delivered

def step(
    self,
    throttle_cmd: float,
    brake_cmd: float,
    steer_angle_rad: float,
    manual_override: bool,
    mode_x_aero: bool,
    track_curvature: float = 0.0
) -> dict[str, float]:
    # 1. Aerodynamic Loads
    cd = self.p.cd_mode_x if mode_x_aero else self.p.cd_mode_z
    cl = self.p.cl_mode_x if mode_x_aero else self.p.cl_mode_z

    f_drag = 0.5 * self.p.rho_air * cd * self.p.frontal_area * (self.vx ** 2)
    f_downforce = 0.5 * self.p.rho_air * cl * self.p.frontal_area * (self.vx ** 2)

    m_total = self.p.mass_empty + self.p.mass_driver + self.fuel_kg
    f_z_total = m_total * 9.81 + f_downforce
    f_rr = self.p.crr * f_z_total

    # 2. Battery Thermal Derate
    if self.t_core_bat > self.p.t_bat_warn:
        derate = 1.0 - (self.t_core_bat - self.p.t_bat_warn) / (self.p.t_bat_crit - self.p.t_bat_warn)
        self.derate_factor = float(np.clip(derate, 0.0, 1.0))
    else:
        self.derate_factor = 1.0

    # 3. Powertrain Arbitration
    p_mguk_reg_limit = self.evaluate_2026_mguk_power_limit(self.vx, manual_override)
    p_mguk_avail = p_mguk_reg_limit * self.derate_factor

```

```

if self.soc <= 0.05:
    p_mguk_avail = 0.0

p_ice_dem = throttle_cmd * self.p.p_ice_max
p_mguk_dem = throttle_cmd * p_mguk_avail

p_regen_mech_req = 0.0
if brake_cmd > 0.0 and self.soc < 0.98 and self.e_lap_regen < self.p.e_regen_lap_max:
    p_brake_total = brake_cmd * 800000.0
    p_regen_mech_req = -min(p_brake_total * 0.55, self.p.p_mguk_max)

p_mech_req = p_mguk_dem if throttle_cmd > 0.0 else p_regen_mech_req

# 4. Electrical Battery Resolution
v_oc = self.compute_open_circuit_voltage(self.soc, self.t_core_bat)
r0 = self.compute_internal_resistance(self.soc, self.t_core_bat)

i_delivered, v_term, p_mguk_delivered = self.solve_battery_electrical(
    p_mech_req, v_oc, r0
)

self.current_delivered = i_delivered
self.voltage_terminal = v_term

if p_mguk_delivered >= 0.0:
    self.p_deploy_delivered = p_mguk_delivered
    self.p_regen_delivered = 0.0
else:
    self.p_deploy_delivered = 0.0
    self.p_regen_delivered = -p_mguk_delivered
    self.e_lap_regen += (-p_mguk_delivered * self.dt)

# 5. Tyre Thermo-Mechanical Adhesion
mu_thermal = np.exp(
    -((self.t_tyre_tread - self.p.t_tyre_opt) ** 2) / (2.0 * (self.p.tyre_thermal_sigma ** 2))
)
load_sens = 1.0 / (1.0 + self.p.k_load_sensitivity * (f_z_total / self.p.fz_ref))
wear_penalty = max(0.0, 1.0 - 0.35 * self.tyre_wear)

mu_x_eff = self.p.mu_x_nominal * mu_thermal * load_sens * wear_penalty
mu_y_eff = self.p.mu_y_nominal * mu_thermal * load_sens * wear_penalty

p_drive_total = p_ice_dem + self.p_deploy_delivered

```

```

f_prop_raw = (p_drive_total * self.p.eta_drivetrain) / max(self.vx, 1.0)
f_brake_raw = brake_cmd * 25000.0
f_net_longitudinal = f_prop_raw - f_brake_raw

f_x_max = mu_x_eff * f_z_total
f_y_demand = m_total * (self.vx ** 2) * track_curvature
f_y_max = mu_y_eff * f_z_total

ellipse_norm = (f_net_longitudinal / f_x_max) ** 2 + (f_y_demand / f_y_max) ** 2
if ellipse_norm > 1.0:
    scale = 1.0 / np.sqrt(ellipse_norm)
    f_x_delivered = f_net_longitudinal * scale
    f_y_delivered = f_y_demand * scale
else:
    f_x_delivered = f_net_longitudinal
    f_y_delivered = f_y_demand

# 6. Chassis Acceleration and Curvilinear Kinematics
dv_x = (f_x_delivered - f_drag - f_rr) / m_total
self.vx = max(0.1, self.vx + dv_x * self.dt)

denom = max(0.001, 1.0 - self.n * track_curvature)
ds = (self.vx * np.cos(self.heading_error)) / denom
dn = self.vx * np.sin(self.heading_error)
yaw_rate = self.vx * track_curvature
d_heading_error = yaw_rate - track_curvature * ds

self.s += ds * self.dt
self.n += dn * self.dt
self.yaw += yaw_rate * self.dt
self.heading_error += d_heading_error * self.dt
self.x_map += self.vx * np.cos(self.yaw) * self.dt
self.y_map += self.vx * np.sin(self.yaw) * self.dt

# 7. Battery State Integration
q_pack_as = self.p.q_cell_nominal_ah * 3600.0 * self.p.cells_parallel * self.soh_capacity
self.soc = float(np.clip(self.soc - (i_delivered / q_pack_as) * self.dt, 0.0, 1.0))

d_v1 = (-self.v1 / (self.p.r1_polarization * self.p.c1_polarization)
        + i_delivered / self.p.c1_polarization) * self.dt
self.v1 += d_v1

# Bernardi Heat Generation

```

```

q_joule = (i_delivered ** 2) * r0
q_pol = (self.v1 ** 2) / self.p.r1_polarization
t_core_k = self.t_core_bat + 273.15
q_entropic = -i_delivered * t_core_k * (self.p.dvoc_dt * self.p.cells_in_series)
q_gen_total = q_joule + q_pol + q_entropic

q_core_to_surf = (self.t_core_bat - self.t_surf_bat) / self.p.r_core_surf_bat
q_surf_to_cool = (self.t_surf_bat - self.p.t_coolant_bat) / self.p.r_surf_cool_bat

self.t_core_bat += ((q_gen_total - q_core_to_surf) / self.p.c_core_bat) * self.dt
self.t_surf_bat += ((q_core_to_surf - q_surf_to_cool) / self.p.c_surf_bat) * self.dt

# Capacity and Resistance Ageing
arrhenius_soh = np.exp((3500.0 / 8.314) * (1.0 / 298.15 - 1.0 / t_core_k))
d_soh = -(abs(i_delivered) * self.dt) / (2.0 * 1500.0 * q_pack_as) * arrhenius_soh
self.soh_capacity = max(0.60, self.soh_capacity + d_soh)
self.soh_resistance = min(2.0, self.soh_resistance - 1.2 * d_soh)

# 8. Tyre Thermal and Wear Integration
p_slip = abs(f_x_delivered * (self.vx * 0.035)) + abs(f_y_delivered * (self.vx *
np.sin(abs(steer_angle_rad))))
q_cond = (self.t_tyre_tread - self.t_tyre_carcass) / self.p.r_tyre_cond
h_conv = 15.0 + 1.8 * (self.vx ** 0.8)
q_conv = h_conv * 0.085 * (self.t_tyre_tread - 25.0)

self.t_tyre_tread += ((p_slip - q_cond - q_conv) / self.p.c_tyre_tread) * self.dt
self.t_tyre_carcass += ((q_cond - 0.05 * (self.t_tyre_carcass - 25.0)) / self.p.c_tyre_carcass) *
self.dt

temp_ratio = max(1.0, self.t_tyre_tread / self.p.t_tyre_opt)
d_wear = self.p.kappa_wear_rate * p_slip * (temp_ratio ** self.p.gamma_wear_temp) *
self.dt
self.tyre_wear = float(np.clip(self.tyre_wear + d_wear, 0.0, 1.0))

# Fuel and Energy Accounting
self.fuel_kg = max(0.0, self.fuel_kg - p_ice_dem * 0.000055 * self.dt)
self.delta_energy_wh = (-i_delivered * v_term * self.dt) / 3600.0

return {
    "speed_mps": self.vx,
    "speed_kmh": self.vx * 3.6,
    "track_s_m": self.s,
    "track_n_m": self.n,

```

```

    "heading_error_rad": self.heading_error,
    "pos_x_m": self.x_map,
    "pos_y_m": self.y_map,
    "battery_soc": self.soc,
    "battery_soh_capacity": self.soh_capacity,
    "battery_soh_resistance": self.soh_resistance,
    "battery_t_core_c": self.t_core_bat,
    "battery_t_surf_c": self.t_surf_bat,
    "battery_i_delivered_a": self.current_delivered,
    "battery_v_terminal_v": self.voltage_terminal,
    "battery_power_deploy_kw": self.p_deploy_delivered / 1000.0,
    "battery_power_regen_kw": self.p_regen_delivered / 1000.0,
    "battery_thermal_derate": self.derate_factor,
    "delta_energy_wh": self.delta_energy_wh,
    "tyre_t_tread_c": self.t_tyre_tread,
    "tyre_t_carcass_c": self.t_tyre_carcass,
    "tyre_wear_fraction": self.tyre_wear,
    "tyre_life_remaining": max(0.0, 1.0 - self.tyre_wear),
    "tyre_grip_multiplier": float(mu_thermal * wear_penalty),
    "lap_regen_mj": self.e_lap_regen / 1e6,
    "fuel_mass_kg": self.fuel_kg,
    "lap_time_loss_sec": 2.8 * self.tyre_wear + 1.2 * (self.tyre_wear ** 2)
}

```

```

def extract_relative_speed_gap(
    ego_telemetry: dict[str, float], opponent_telemetry: dict[str, float]
) -> dict[str, float]:
    """
    Computes track-progress spatial offsets and closing dynamics between
    ego telemetry and time-aligned opponent tracking estimates.
    """

    spatial_gap = opponent_telemetry["track_s_m"] - ego_telemetry["track_s_m"]
    speed_gap = ego_telemetry["speed_mps"] - opponent_telemetry["speed_mps"]
    closing_speed = speed_gap * np.cos(ego_telemetry["heading_error_rad"])

    return {
        "spatial_gap_m": spatial_gap,
        "speed_gap_mps": speed_gap,
        "closing_speed_mps": closing_speed,
        "heading_delta_rad": ego_telemetry["heading_error_rad"] -
        opponent_telemetry["heading_error_rad"]
    }

```

The mathematical coupling established across this simulation kernel reveals the structural vulnerabilities of naive tactical deployment. Deploying the full 350 kW electrical boost without monitoring core-to-surface battery heat transfer drives internal cell temperatures into derating regimes within successive straights, reducing subsequent harvest efficiency and inducing substantial throttle clipping. Simultaneously, the resultant longitudinal tyre slip power elevates tread surface flash temperatures past the optimal adhesion window, inducing compound degradation that penalizes downstream stint pace by tenths of a second per lap. High-performance strategy optimization therefore depends on evaluating overtakes not as isolated spatial opportunities, but as multi-lap thermodynamic trade-offs where finite electrical reserves, regulatory recovery limits, and contact-patch health dictate net finishing position.

Works cited

1. STREAMLIT_2D_F1_SIMULATOR_PLAN.md
2. ENERGY_OVERTAKE_SYSTEM_DESIGN.md
3. Schematic of the equivalent circuit model. - ResearchGate, https://www.researchgate.net/figure/Schematic-of-the-equivalent-circuit-model_fig1_260215082
4. Tire/road interaction models - MegaRide, <https://www.megaride.eu/wp-content/uploads/2021/01/TireTechnologyInternational2014-Annual-Review.pdf>
5. F1 2026 new cars new rules - F1 Monaco Grand Prix 2027, <https://www.ticketgrandprix.com/f1-2026-new-cars-new-rules/>
6. F1 | 2026 Regulations, the FIA changes technical terminology, <https://f1.newsauto.it/en/formula-1/f1-2026-regulations-the-fia-changes-technical-terminology-11603>
7. Pacejka Magic Formula, <http://www.racer.nl/reference/pacejka.htm>
8. 2026 Formula 1: The Convergence of Hybrid Power and Cognitive, <https://www.axiorablogs.com/blog/2026-formula-1-the-convergence-of-hybrid-power-and-cognitive-strategy>
9. FIA 2026 F1 Regulations - Section C [Technical] - Iss 20, https://api.fia.com/system/files/documents/fia_2026_f1_regulations_-_section_c_technical_-_iss_20_-_2026-08-05.pdf
10. F1 in 2026: The "Revolution" We've been waiting for?, <https://www.thedrivershub.com/f1-in-2026-the-revolution-weve-been-waiting-for/>
11. CeritaF1 | Wanfarid93.com - ENGINEERING, <https://wanfarid93.com/CeritaF1/>
12. How will the "Override" boost mode of the F1 2026 regulations work?, <https://www.motorsinside.com/en/f1/news/33753-how-will-override-boost-mode-of-f1-2026-regulations-work>
13. Schematic of the two-state thermal model (adapted from Ref. [20]), https://www.researchgate.net/figure/Schematic-of-the-two-state-thermal-model-adapted-from-Ref-20_fig2_260215082
14. An Electro-Thermal Model for LFP Cells - PSE Community.org,

<https://psecommunity.org/wp-content/plugins/wpor/includes/file/2303/LAPSE-2023.14414-1v1.pdf>

15. [unknown_url](#)
16. Smart Core and Surface Temperature Estimation Techniques for, <https://www.mdpi.com/1996-1073/15/2/623>
17. Deep Koopman Observer for Lithium-Ion Battery Temperature, <https://www.mdpi.com/2032-6653/17/6/310>
18. The Battery Design Module User's Guide - COMSOL Documentation, <https://doc.comsol.com/6.4/doc/com.comsol.help.battery/BatteryDesignModuleUsersGuide.pdf>
19. Thermal state monitoring of lithium-ion batteries Progress, <https://vbn.aau.dk/ws/portalfiles/portal/770123859/1-s2.0-S0360128523000503-main.pdf>
20. friction ellipse | DrRacing's Blog, <https://drracing.wordpress.com/tag/friction-ellipse/>
21. Discrete Tire Model Application for Vehicle Dynamics Performance, https://vtechworks.lib.vt.edu/bitstream/handle/10919/74394/Siramdasu_Y_D_2015.pdf?isAllowed=y&sequence=1
22. (PDF) Analysis of the results provided by a grip and thermodynamics, https://www.researchgate.net/publication/268808623_Analysis_of_the_results_provided_by_a_grip_and_thermodynamics-sensitive_tireroad_interaction_force_characterization_procedure